SC4050 Lab 4: MPI Fundamentals
Week 8

Lab Objectives

By the end of this lab, you will be able to:

- Measure point-to-point communication time correctly (warm-up, repeated trials) and fit the cost model $T(n) = \alpha + n\beta$ to your measurements.
- Use message tags and wildcard receives (`MPI_ANY_SOURCE`, `MPI_ANY_TAG`) to handle messages whose sender and arrival order are not known in advance, and recover the true sender/tag afterward from `MPI_Status`.
- Use non-blocking communication (`MPI_Isend`/`MPI_Irecv`/`MPI_Waitall`) to *overlap* communication with computation.
- Implement a collective reduction by hand, using only point-to-point calls, and explain why a library implementation (next week's `MPI_Allreduce`) is worth having.

Overview and Setup

This lab carries **5 marks total**: four hands-on tasks. There is no file to submit and no separate quiz this week. A TA or the instructor will check off each task when ready, based on what runs successfully on your screen and what you can explain when asked. Keep your terminal session and source files open until you have been checked off for every task.

Before starting Task 1, confirm your environment works: write and run a trivial 2-node, 2-rank MPI program that has each rank print its hostname and rank, so you can confirm the two ranks really landed on two different physical nodes. You could use either your Raspberry cluster (recommended) or Aspire2A.

You can find the four skeleton files for the tasks in NTULearn. You will fill in and complete that code for each task.

- `pingpong_skeleton.c`
- `collector_skeleton.c`
- `overlap_skeleton.c`
- `allreduce_skeleton.c`

Task 1: Ping-Pong Benchmark (1 mark)

Open `pingpong_skeleton.c`. It measures round-trip time between rank 0 and rank 1 for a range of message sizes, but three parts are left as `TODO` comments for you to fill in:

1. A short, untimed warm-up loop before timing starts (so connection setup doesn't pollute your first real measurement).

2. The timed round-trip loop itself, using `MPI_Wtime()`.
3. A two-point fit of your measurements to $T(n) = \alpha + n\beta$, using your smallest and largest tested message sizes, to estimate α, β, the implied bandwidth $1/\beta$, and the crossover point $n^* = \alpha/\beta$.

Run with exactly 2 ranks, one per node (the same multi-node setup as your sanity check above, not two ranks on the same node, otherwise you are measuring shared memory, not a real network).

Checkpoint: the program compiles and runs across 2 nodes, prints a table of one-way times across all 7 sizes, and prints fitted α, β, bandwidth, and n^*.

Task 2: Tags & Wildcard Receives (1 mark)

Open `collector_skeleton.c` (do not modify the worker branch). Rank 0 is a *collector*: every other rank sends it exactly one message, but each worker waits a random, short amount of time first, so messages do not arrive at rank 0 in rank order. Rank 0 must not assume who is next.

1. Post a receive that accepts a message from *any* source and *any* tag, using `MPI_ANY_SOURCE` and `MPI_ANY_TAG`: you do not know who arrives next.
2. Recover the *true* sender and tag afterward from the `MPI_Status` that `MPI_Recv` fills in, and use them to identify who you just heard from.
3. Sanity-check each message: make sure no rank reports twice, and that the payload matches what that rank should have sent.

Run with at least 4 ranks on at least 2 nodes.

Checkpoint: the program compiles, prints which rank it received each message from (recovered via `status`), prints `COLLECTOR: PASS`, and, if you run it two or three times in a row, we can see the arrival order change between runs.

Task 3: Overlap Communication with Computation (1.5 marks)

Open `overlap_skeleton.c`. Every rank exchanges a block of data with its two ring neighbors, and separately does a fixed amount of CPU-bound local computation (`burn_cpu`, already written for you). You will compare two strategies for doing both:

- **Sequential**: communicate fully (two `MPI_Sendrecv` calls), *then* compute.
- **Overlapped**: *start* the same four transfers as `MPI_Isend` and `MPI_Irecv` calls, compute *while in flight*, then `MPI_Waitall`.

Fill in the two TODOs (sequential exchange, then overlapped exchange); the timing and averaging code is already written.

Run with at least 4 ranks on at least 2 nodes. *Note*: the exact savings you see will vary by machine and by run. A positive savings of even 5–15% is a successful result; do not worry if one single run looks flat, run it again.

Checkpoint: the program compiles, prints both average times and a savings figure, and the overlapped strategy is not slower than the sequential one.

Task 4: Ring All-Reduce by Hand (1.5 marks)

Open `allreduce_skeleton.c`. Every rank holds a local value (rank 0 holds 1.0, rank 1 holds 2.0, and so on). Using *only* point-to-point communication, no `MPI_Reduce` or `MPI_Allreduce`, those are next week's material, compute the global sum of every rank's local value, so that *every* rank ends up knowing it, not just rank 0.

The pattern (a *ring all-reduce*): run exactly $P - 1$ rounds, where P is the number of ranks. In each round:

1. Exchange a value called `relay` with your neighbors using one `MPI_Sendrecv` call: send `relay` to your right neighbor, receive into a new variable `incoming` from your left neighbor.
2. Add `incoming` to your running total.
3. Set `relay = incoming`, so next round you forward what you just received, not your own original value again, continuing the relay around the ring.

Checkpoint: every rank prints a running total matching the printed expected value, and `PASS`, at more than one rank count (e.g. 4, 5, and 9).

Mark Summary

Task	**Marks**
Task 1: Ping-Pong Benchmark	1.0
Task 2: Tags & Wildcard Receives	1.0
Task 3: Overlap Communication with Computation	1.5
Task 4: Ring All-Reduce by Hand	1.5
Lab total	**5.0**